LOOP

(Live Object-Oriented Playground)

Personal Project 03

Jun Xiang

6th November 2026

Table of Contents

1. Overview 3

1.1 Problem 3

1.2 Solution 3

1.3 Goals 4

1.4 Features 4

2. Scope 4

2.1 In Scope 4

2.2 Out of Scope 5

3.1 Functional Requirements 6

3.2 Non-Functional Requirements 7

4. Methodology 8

4.1 Technology Stack 8

4.2 Running the Project 8

4.3 Application Structure 8

4.4 Main OOP Design 9

4.5 OOP Relationships 10

4.6 Execution Flow 10

4.7 Scenario Design 11

4.8 Design Decisions 11

4.9 Interactive OOP Fundamentals 12

5. Limitations & Risks 12

6. Testings and Findings 13

6.1 Automated Testing 13

6.2 Test Case 1: Inheritance and Method Execution 14

6.3 Test Case 2: Polymorphism 15

6.4 Test Case 3: Method Hover Preview 16

6.5 Test Case 4: Playback and Automatic Scrolling 16

6.6 Summary of Findings 17

7. Challenges 17

8. Improvements 19

1. Overview

Last Updated: 6 November 2026

LOOP (Live Object-Oriented Playground) is an interactive Python OOP visualizer designed to make object-oriented programming concepts easier to understand through visual execution.

It uses three scenarios: Cybersecurity, Library, and Smart Home, where users can interact with different objects and methods. As Python methods execute, LOOP visually shows method calls, object interactions, state changes, inheritance, and the corresponding source code step by step.

1.1 Problem

Object-oriented programming concepts such as classes, objects, inheritance, encapsulation, and object interactions can be difficult to understand from source code alone.

Beginners can see what the code says, but may not clearly understand which object is running a method, how objects interact, or how their states change during execution.

1.2 Solution

LOOP provides an interactive environment for exploring Python OOP concepts through scenario-based simulations.

Users can select a scenario, execute methods, and follow what happens through visual explanations.

LOOP allows users to:

- execute real Python methods through FastAPI.
- follow source-code highlighting during execution.
- see object instances and their state changes.
- visualize method calls through animated circuits.
- explore method relationships by hovering over methods.
- observe inheritance and calls using `super()`.
- follow execution through step-by-step Terminal explanations.
- control playback using Play, Pause, Previous, Next, Replay, and Reset
- see automatic scrolling and a compact sticky Terminal to follow the executing code

All scenarios are simulations and do not interact with real-world systems.

1.3 Goals

LOOP aims to:

- make Python OOP concepts easier to understand visually.
- clearly distinguish classes from object instances.
- show method calls and object interactions during execution.
- demonstrate encapsulation, inheritance, composition, abstraction, and polymorphism.
- keep Python as the source of truth for simulation state.
- provide an interactive and easy-to-follow learning experience.
- support different scenarios that demonstrate OOP concepts in various contexts.

1.4 Features

- Three interactive scenarios: Cybersecurity, Library, and Smart Home.
- Real Python execution through FastAPI.
- Interactive source-code display with execution highlighting.
- Animated circuits showing method calls and object relationships.
- Method-hover previews without executing code.
- Object-instance indicators and live state changes.
- Step-by-step Terminal with Story, Code, Concept, and Changed explanations.
- Action summaries that reflect successful and unsuccessful outcomes.
- Interactive OOP Fundamentals with expandable explanations and scenario examples.
- Previous, Next, Play, Pause, Replay, and Reset controls.
- Automatic scrolling and a compact sticky Terminal.
- Optional line numbers and a visualization legend.
- Light and dark themes with responsive layouts.
- Keyboard navigation and reduced-motion support.

2. Scope

2.1 In Scope

Simulation

- Three interactive scenarios: Cybersecurity, Library, and Smart Home.
- Predefined actions that execute real Python methods.
- Object interactions and state changes based on the selected actions.
- Repeatable simulations with independent object states.

Object-Oriented Programming

- Classes and object instances.

- Encapsulation and object state management.
- Composition and relationships between objects.
- Abstraction and abstract methods.
- Inheritance, method overriding, and `super()`.
- Polymorphism and shared method interfaces.
- Properties and controlled attribute access.

Visualization

- Real Python source-code display.
- Source-line highlighting during execution.
- Animated circuits showing method calls and relationships.
- Hover previews of connected methods.
- Object-instance indicators and state changes.
- Terminal history with execution events and OOP explanations.

User Interface

- HTML, CSS, and JavaScript frontend.
- Scenario selection and interactive methods.
- Previous, Next, Play, Pause, Replay, and Reset controls.
- Automatic scrolling and a compact sticky Terminal.
- Optional line numbers and a visualization legend.
- Light and dark themes.
- Responsive layouts for desktop and mobile devices.

2.2 Out of Scope

- Interaction with real-world systems or external devices.
- User-written Python code or custom scenarios.
- Full Python debugging or interpreter-level tracing.
- User accounts, authentication, or progress tracking.
- Database storage or persistent simulation states.
- Multiplayer or collaborative features.
- AI or machine learning.
- Game mechanics such as scores or win/loss conditions.

3.1 Functional Requirements

No.	Functional Requirement	Description
FR-01	Scenario Selection	Users can switch between Cybersecurity, Library, and Smart Home scenarios.
FR-02	Interactive Methods	Users can select predefined Python methods to execute actions.
FR-03	Python Execution	Selected actions execute real Python methods through FastAPI.
FR-04	Source Code Display	Users can view the actual Python source code for each scenario.
FR-05	Execution Highlighting	Executing, waiting, previously executed, and unrelated code are visually distinguished.
FR-06	Circuit Visualization	Animated circuits show method calls, inheritance, and property access.
FR-07	Method Hover Preview	Users can hover over methods to highlight their connected circuits without executing code.
FR-08	Object Instances	Object instances are displayed separately from their classes, with indicators showing their current states.
FR-09	State Changes	Changes to object attributes are displayed during execution.
FR-10	Execution Terminal	The Terminal shows execution history, code explanations, and relevant OOP concepts.
FR-11	Playback Controls	Users can play, pause, step forward, step backward, replay, and reset an execution.
FR-12	Automatic Scrolling	The interface can automatically follow executing code, with an option to disable this feature.
FR-13	Compact Terminal	A compact Terminal remains accessible when the original Terminal scrolls out of view.

FR-14	Display Settings	Users can toggle source-code line numbers, switch themes, and view a legend explaining visual indicators.
FR-15	Interactive OOP Fundamentals	Users can select an OOP concept to expand its explanation and view an example from LOOP, with smooth transitions and navigation between concepts.
FR-16	Context-Aware Explanations	Terminal explanations and action summaries reflect the actual execution result, including successful and unsuccessful actions.
FR-17	Session State Management	Each browser session maintains independent simulation states, which remain available between actions until reset or session expiry.

3.2 Non-Functional Requirements

No.	Non-Functional Requirement	Description
NFR-01	Safety	LOOP must not interact with real-world systems or external devices through its simulations.
NFR-02	Simplicity	The interface should use clear labels, intuitive controls, and short explanations.
NFR-03	Readability	Source code, visual indicators, and explanations should be easy to understand in both themes.
NFR-04	Accuracy	Visualized execution and object states must reflect the actual Python simulation.
NFR-05	Performance	Animations, playback, and scrolling should run smoothly without unnecessary delays.
NFR-06	Responsiveness	The interface should adapt to desktop and mobile screen sizes.
NFR-07	Accessibility	The interface should support keyboard navigation and reduced-motion preferences where applicable.
NFR-08	Maintainability	Python logic, JavaScript behaviour, and CSS styling should remain clearly separated.

NFR-09	Extensibility	Additional scenarios should be addable without major changes to the existing architecture.
NFR-10	Local Execution	LOOP should require minimal setup to run locally.

4. Methodology

4.1 Technology Stack

- Python: Defines OOP classes, simulation logic, object states, and execution tracing.
- FastAPI: Connects the Python backend to the browser.
- HTML: Provides the structure of the interface.
- CSS: Handles layout, themes, highlighting, and animations.
- JavaScript: Handles user interactions, playback, circuits, and source-code visualization.

4.2 Running the Project

LOOP requires Python 3.11 or newer.

From the project folder, install the required dependencies: `python -m pip install -r` requirements.txt

Start the application: `python -m uvicorn api:app --reload`

Then open `http://127.0.0.1:8000` in a browser.

4.3 Application Structure

LOOP separates Python execution from the browser visualization.

The main execution flow is: Python Scenario → Execution Tracer → FastAPI → JavaScript Playback → Browser Visualization

- Python contains the actual object states and methods.
- The execution tracer records method calls, executed lines, and state changes.
- FastAPI sends the execution data to the frontend.
- JavaScript uses this data to update the source-code highlighting, circuits, object indicators, and Terminal.

The project is organized into separate parts:

Component	Purpose
scenarios/	Contains the Python classes, actions, and explanations for each scenario.
engine/	Handles scenario management, source-code information, and execution tracing.
api.py	Provides the API endpoints and manages simulation sessions.
web/	Contains the frontend HTML, CSS, and JavaScript files.
tests/	Contains automated tests for the Python simulation and execution system.
assets/	Contains shared visual assets, including favicon images.
Component	Purpose
scenarios/	Contains the Python classes, actions, and explanations for each scenario.

4.4 Main OOP Design

Each scenario uses different Python classes to demonstrate OOP concepts.

Scenario	Main Classes	Purpose
Cybersecurity	Attacker, Attack, MalwareAttack, RansomwareAttack, InterceptAttack, Device, Connection, Victim	Demonstrates attack and recovery actions, inheritance, abstraction, and polymorphism.
Library	Member, Library, Book	Demonstrates object interactions, object references, properties, and encapsulation.
Smart Home	Resident, HomeHub, SmartDevice, Light, Fan, Thermostat, DoorLock	Demonstrates method delegation, inheritance, overriding, and polymorphism.

LOOP also uses shared backend components:

Component	Purpose
Tracer	Records method calls, executed source lines, and object state changes.
Scenario Manager	Loads scenarios, executes selected actions, and prepares data for the frontend.
Source Reader	Retrieves actual Python source code, class information, and line numbers.

4.5 OOP Relationships

LOOP demonstrates several OOP concepts through the three scenarios:

- Encapsulation: Objects manage their own attributes through methods.
- Abstraction: The abstract `Attack` class defines a common `execute()` method.
- Inheritance: Classes such as `RansomwareAttack` and `Light` inherit from parent classes.
- Method overriding: Child classes provide their own implementations of inherited methods.
- Polymorphism: Different objects respond to the same method call using their own implementations.
- `super()`: Child methods reuse behaviour defined in their parent classes.
- Object relationships: Objects store references to and interact with other objects.
- Properties: The `@property` decorator provides controlled access to calculated values.
- These concepts are demonstrated through actual Python execution rather than predefined visual animations alone.

4.6 Execution Flow

When a user selects an executable method:

1. JavaScript sends the selected action and target to FastAPI.
2. FastAPI runs the corresponding Python method.
3. The tracer records the executed lines, method calls, and object state changes.
4. The backend returns the recorded execution steps to the browser.
5. JavaScript plays the steps in sequence.
6. The source code, circuits, and object indicators update to reflect each step.
7. The Terminal explains what happened and the OOP concepts involved.

Users can control the playback using Previous, Next, Play, Pause, Replay, and Reset.

Automatic scrolling can follow the executing code, while a compact sticky Terminal keeps the explanations visible when the original Terminal is out of view.

4.7 Scenario Design

LOOP currently includes three scenarios:

Cybersecurity

- Simulates fictional attacks and recovery actions involving devices and connections.
- Demonstrates abstraction, inheritance, method overriding, super(), and polymorphism.

Library

- Simulates borrowing, returning, renewing, and checking books.
- Demonstrates object references, method delegation, properties, and encapsulation.

Smart Home

- Simulates controlling lights, fans, thermostats, and door locks through a home hub.
- Demonstrates inheritance, method overriding, polymorphism, and controlled state changes.

Each scenario defines its own classes, objects, available actions, and Terminal explanations.

All scenarios use the same execution tracer and visualization system, allowing new scenarios to be added without rebuilding the playground.

4.8 Design Decisions

- Python as the source of truth: Object states and execution results come from real Python methods rather than frontend simulations.
- Real execution tracing: The tracer records actual Python source lines and method calls.
- Shared visualization system: All scenarios use the same playback, highlighting, circuit, and Terminal components.
- Separate classes and objects: The interface displays Python classes alongside their object instances.
- Consistent circuit colours: Blue represents normal method calls, purple represents `super()` calls, and amber dashed lines represent property access.
- Step-by-step playback: Users can control execution speed and review earlier steps without rerunning the action.
- Persistent object states: Object changes are retained between actions within the same scenario session until Reset.
- Extensible scenarios: New scenarios can be added through separate Python modules while reusing the existing frontend.

4.9 Interactive OOP Fundamentals

LOOP includes an interactive Fundamentals section introducing nine OOP concepts: Classes and Objects, Attributes, Properties, Methods, Inheritance, Encapsulation, Polymorphism, Abstraction, and Object Relationships.

Each concept initially displays a short description. Selecting a concept expands it into a focused view containing:

- A beginner-friendly explanation.
- An example from LOOP's scenarios.
- A link to the relevant Python documentation.
- Previous, Next, and Collapse controls.

The selected concept grows into the expanded view through a smooth transition, while other concepts are temporarily hidden.

Users can return to the overview by collapsing the concept, clicking outside, or pressing Escape when appropriate.

The design retains the website's flat layout and supports smaller screens, keyboard navigation, and reduced-motion preferences.

5. Limitations & Risks

- The scenarios are simplified examples and may not fully represent real-world systems.
- Users can only execute predefined actions and cannot write or run their own Python code.
- LOOP focuses on selected OOP concepts rather than covering all Python features.
- The execution tracer records relevant Python events rather than providing full interpreter-level debugging.
- Python actions execute before the browser plays back the recorded steps, so animation timing does not represent actual execution speed.
- Complex method relationships and multiple visual indicators may be difficult for beginners to follow.
- Long classes and frequent automatic scrolling may make execution harder to follow, especially on smaller screens.
- The frontend must remain synchronized with the recorded Python execution and object states.
- The interface and explanations may need further refinement based on user feedback.

6. Testings and Findings

6.1 Automated Testing

LOOP uses pytest to verify the Python simulation, execution tracing, and application behaviour.

The automated tests cover:

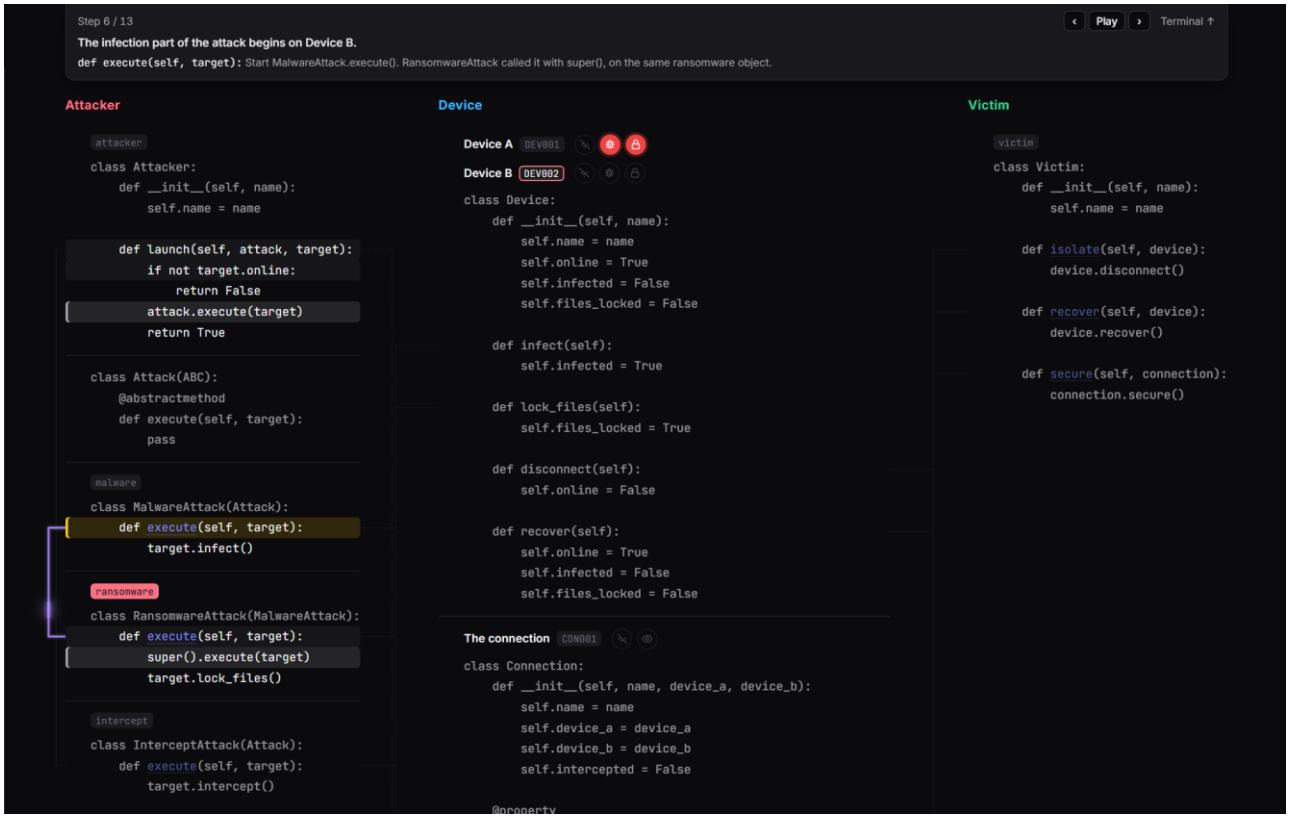
- Scenario loading and action execution.
- Method calls, inheritance, and object interactions.
- Object state changes and reset behaviour.
- Execution tracing and source-code information.
- Successful and unsuccessful action summaries.
- Story accuracy, dynamic object names, and placeholders.
- OOP concept explanations.
- API responses and frontend asset availability.

Additional browser checks were performed to review the interface, playback controls, responsive layouts, animations, and keyboard interactions.

6.2 Test Case 1: Inheritance and Method Execution

Test Input	Expected Result	Findings
Run the Ransomware action in Cybersecurity, targeting Device B.	The child method should call `MalwareAttack.execute()` through `super()`. The execution should be reflected in the source-code highlighting, circuits, and Terminal.	• Parent method was executed and visualized correctly. • Terminal explained the inheritance relationship, and the affected object's state changed accordingly.

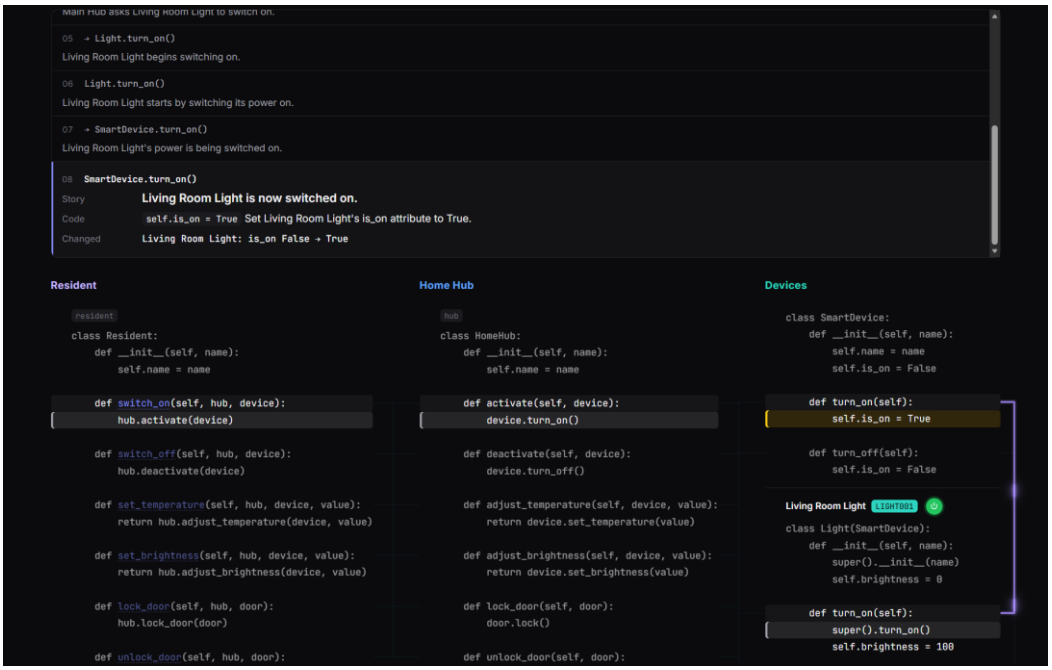
Test Evidence

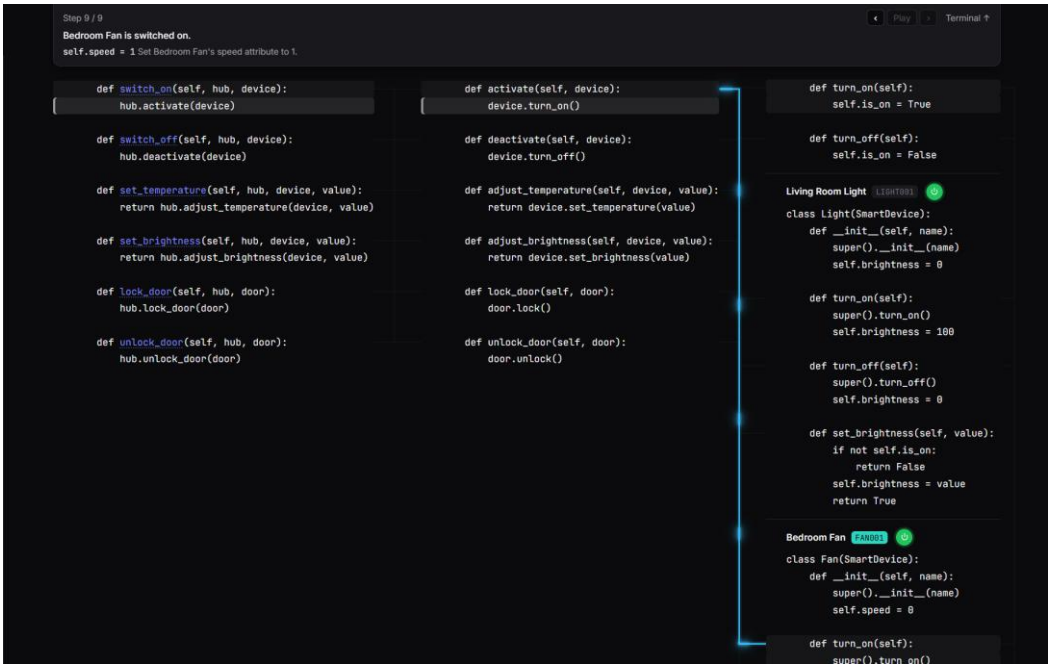


6.3 Test Case 2: Polymorphism

Test Input	Expected Result	Findings
Run the Switch On action for Living Room Light and Bedroom Fan in Smart Home.	`HomeHub.activate()` calls `turn_on()` on each device. Both classes override the same method but perform different actions, demonstrating polymorphism.	The method calls were visualized correctly. The Light was switched on with brightness 100, while the Fan was switched on with speed 1.

Test Evidence

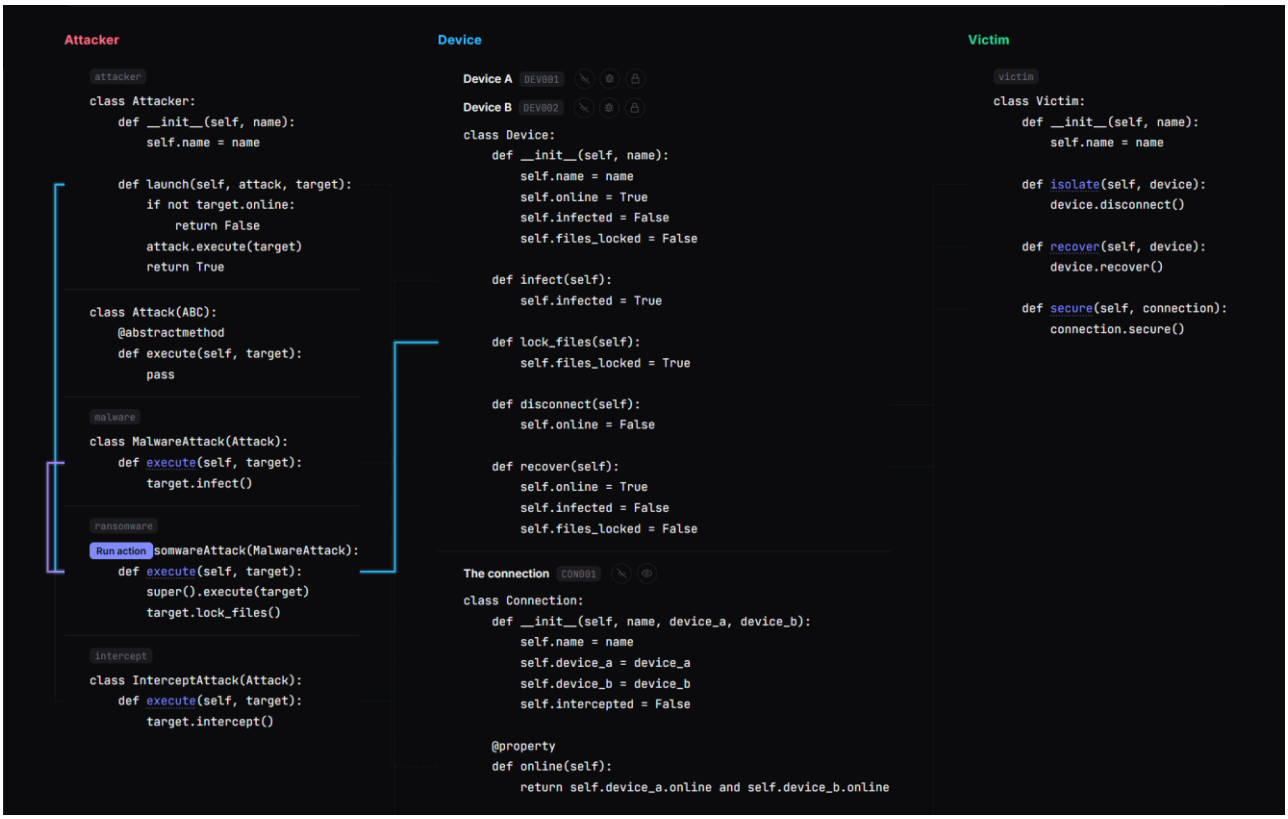




6.4 Test Case 3: Method Hover Preview

Test Input	Expected Result	Findings
Hover over a Python method with known connections.	Connected circuits should illuminate without executing Python code or changing object states.	• Connected circuits displayed a stationary glow. • Terminal and execution state remained unchanged.

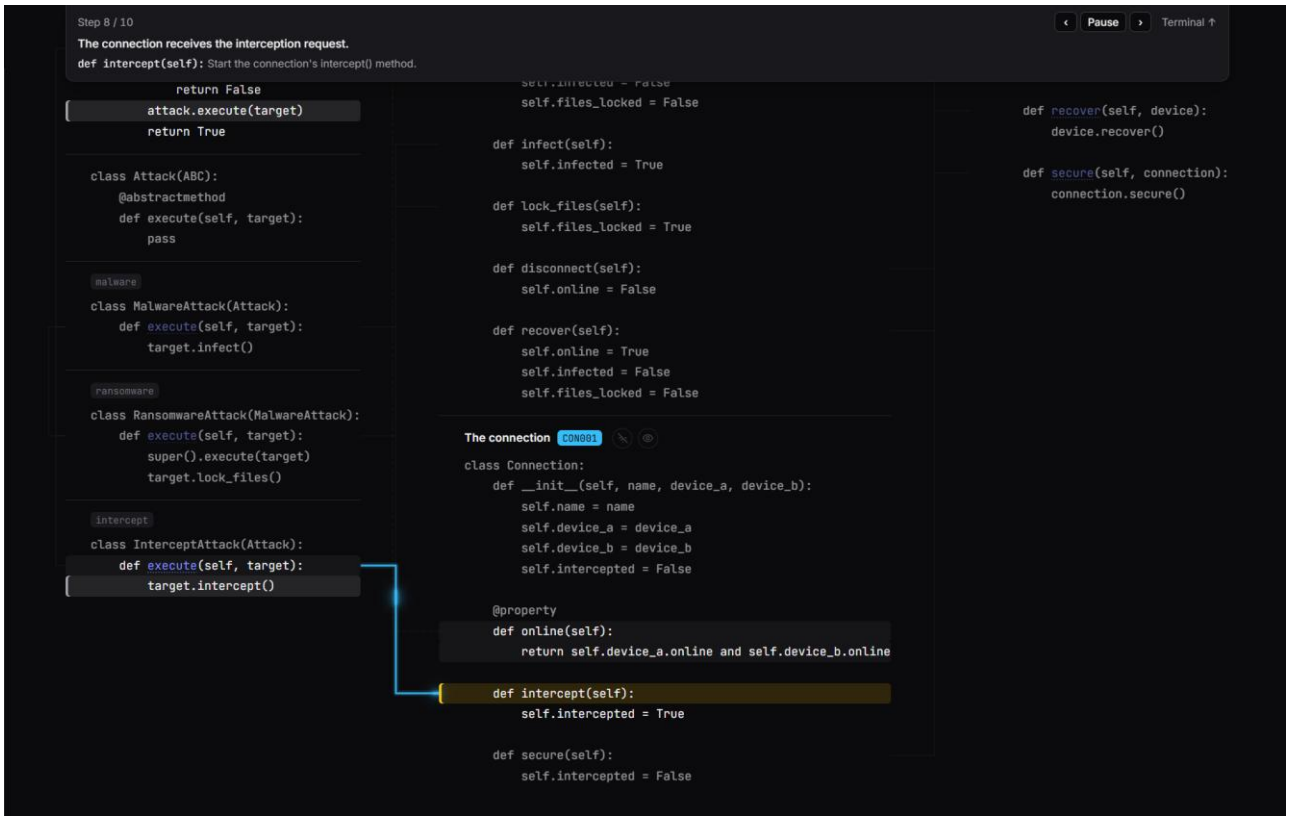
Test Evidence



6.5 Test Case 4: Playback and Automatic Scrolling

Test Input	Expected Result	Findings
Start an action, use the playback controls, and manually scroll during execution.	The interface should follow the executing code, display the compact Terminal when needed, and stop auto-scrolling when the user scrolls manually.	Execution highlighting and playback remained synchronized. Automatic scrolling followed the active code, while manual scrolling paused the following behaviour until resumed.

Test Evidence



6.6 Summary of Findings

The automated tests and browser checks indicate that LOOP's main features are functioning as intended.

The three scenarios demonstrate different OOP concepts through real Python execution, while the source-code highlighting, circuits, and Terminal help users follow method calls and object state changes.

The interface was also checked in light and dark modes across desktop and mobile screen sizes. No major functional errors were reported during the final checks.

7. Challenges

Challenge	How it was handled
Frontend animations were initially separate from the real Python simulation.	FastAPI endpoints and execution tracing were used so the interface follows actual Python execution events.

Keeping playback controls synchronized with execution events was difficult.	Playback state was centralized so Previous, Next, Play, Pause, and Replay use the same recorded event sequence.
Previous playback did not restore source-code highlighting and scrolling correctly.	Playback reconstruction and scrolling logic were improved to follow the selected execution step.
Reset stopped working correctly during playback development.	Reset was reconnected to the backend endpoint, with playback and interface states cleared before rebuilding the simulation.
Supporting multiple scenarios without duplicating the visualization logic was challenging.	A shared execution and visualization system was developed so all three scenarios use the same playback, circuits, and Terminal components.
It was difficult to distinguish classes from object instances and identify which object was affected.	Object-instance indicators were added, and state-change effects were applied to individual objects instead of entire panels.
Inherited method calls using super() were difficult to understand visually.	Parent-method calls were represented through dedicated circuit connections and explained in the Terminal.
Multiple method relationships caused circuits to overlap, making them difficult to follow.	Circuit paths were separated where necessary, with consistent colours and line styles for different relationship types.
Too many colours and highlighting effects made execution confusing.	Code highlighting was simplified into four states, while circuit colours were separated from actor colours.
Code highlighting had inconsistent readability between light and dark modes.	Theme-specific contrast and opacity values were used to keep active code clear and unrelated code readable.
Long source-code sections made it difficult to view the executing code and Terminal explanations together.	Automatic scrolling and a compact sticky Terminal were introduced, with manual scrolling taking priority when users want to explore independently.
The original JavaScript and CSS files became too large to maintain easily.	They were divided into smaller files based on responsibilities such as playback, circuits, source-code display, panels, and scrolling.

8. Improvements

- Adding more interactive scenarios to demonstrate OOP concepts in different contexts.
- Introducing optional guided lessons for beginners.
- Connecting Terminal OOP concept labels directly to their expanded Fundamentals explanations.
- Improving accessibility, particularly screen-reader navigation.
- Further refining mobile layouts and scrolling behaviour.
- Gathering user feedback to improve the clarity of stories, explanations, and visualizations.